Using this framework on your own application

This guide walks through taking the framework and pointing it at a different application, from an empty copy to a green CI run. It assumes you know Playwright and TypeScript. Read ARCHITECTURE.md first if you haven't, because it explains why the code is split the way it is.

The idea in one minute

The code has two kinds of files.

- **Generic**, which you keep as it is: environment handling, the API base class, the base fixtures, tags, the reporting labels, the Dockerfile, lint and format settings.
- **Application-specific**, which you replace: page objects, API clients, test data, fixtures that wire those together, and the tests themselves. Everything about Toolshop lives here.

Most of the work is in the second group. The order below goes from the cheapest changes to the largest.

1. Start from a clean copy

1. Copy the folder (or use the repository as a GitHub template). Delete the `.git` folder and run `git init`, so the new project has its own history.
2. Change the names: `name` and `description` in `package.json`, the report title in `allurerc.mjs`, the copyright holder in `LICENSE`, and the heading in `README.md`.
3. Run `npm ci` and `npx playwright install chromium`.

Don't delete anything yet. It's easier to replace the Toolshop files one by one while the project still builds, using `npm run typecheck` to tell you what is left.

2. Configuration

Everything environment-specific comes from environment variables, read in `src/config/env.ts`.

- Copy `.env.example` to `.env.local` and fill in `UI_URL`, `API_URL` and `APP_NAME`. `APP_NAME` shows in the report as the top-level grouping.
- Credentials are read by role, as `<ROLE>_EMAIL` and `<ROLE>_PASSWORD`. Toolshop has `admin` and `customer`. If your application has other roles, change the `Role` type in `env.ts` (for example `'admin' | 'manager' | 'viewer'`) and add the matching variables. Credentials are read lazily, so a run that never signs in as a role doesn't need them.
- For another environment, create `.env.qa` and run `TEST_ENV=qa npm test`.
- If the application has only a UI, or only an API, set whichever URL you don't use to the other one. Nothing forces you to use both.

In `playwright.config.ts`, check one setting: `testIdAttribute: 'data-test'`. Toolshop marks its elements with `data-test`. If your application uses `data-testid`, `data-cy` or nothing at all, change it here, or lean on `getByRole` and `getByLabel`, which are preferred anyway.

3. Learn how the application authenticates

Do this before writing any code, because every other layer depends on it. Open the application with the browser dev tools and find out:

- **How does the API authenticate?** A bearer token, a cookie, an API key? Toolshop returns a token from a login endpoint and expects `Authorization: Bearer <token>`.
- **How does the browser remember a login?** Toolshop keeps the token in `localStorage` under `auth-token`. Yours may use a cookie or `sessionStorage`.
- **How long does a login last?** Toolshop's tokens expire in 5 minutes, which is why each test signs in through the API instead of sharing one saved session.

You'll use the answers in sections 4 and 6.

4. API layer (`src/api`)

`BaseApiClient` gives you one method, `send`, that makes a request, checks the status and returns typed JSON. A domain client extends it and exposes business-level methods.

```
import type { APIRequestContext } from '@playwright/test';
import { BaseApiClient } from './base-api-client';
import { bearer } from './bearer';
import type { Project } from './types';

export class ProjectsApi extends BaseApiClient {
  constructor(request: APIRequestContext) {
    super(request);
  }

  create(token: string, name: string): Promise<Project> {
    return this.send<Project>('POST', '/projects', { data: { name }, headers: bearer(tok
  }

  /** A project that is already gone is not an error. */
  async delete(token: string, id: string): Promise<void> {
    await this.send<void>('DELETE', `/projects/${id}`, {
      headers: bearer(token),
      expectedStatus: [204, 404],
    });
  }
}
```

Notes:

- Put response shapes in `src/api/types.ts`. Only include the fields you rely on.
- Use `expectedStatus` for negative tests and for cleanup calls that may legitimately return 404.
- Replace `auth-api.ts` with your login call. The rest of the framework only needs it to return a token (or whatever your authentication produces).
- Replace `bearer.ts` if you use a different header.
- Delete the Toolshop clients you don't need (`brands-api`, `carts-api`, `invoices-api`, `payment-api`, `postcode-api`, `products-api`, `reference-api`, `users-api`, `access-api`) once nothing imports them.

5. Test data (`src/data`)

Tests run in parallel against one application, so they must never share or depend on each other's data. The rule used here is: every test creates its own records with unique values, and removes them afterwards.

```
import { randomBytes } from 'node:crypto';

function uniqueSuffix(): string {
  return `${Date.now().toString(36)}${randomBytes(2).toString('hex')}`;
}

export interface ProjectDraft {
  name: string;
}

export function buildProject(): ProjectDraft {
  return { name: `Test Project ${uniqueSuffix()}` };
}
```

Write one builder per entity. If your application validates input in ways that surprise you, handle it in the builder, once. Examples from the Toolshop version: a password that appears in leaked-password lists is rejected, so the builder generates random ones. Product search ignores words shorter than three letters, so product names avoid them.

If tests need a record that can't be deleted (an order, an audited entry), don't create one per test. Create one shared record once per run in `src/config/global-setup.ts`, the way the shared orderable product works. If you don't need that, delete `global-setup.ts` and remove the `globalSetup` line from `playwright.config.ts`.

6. Fixtures: the wiring (`src/fixtures/app.fixture.ts`)

This is the file you'll edit most. It exposes everything a test can ask for by name. The Toolshop version is long, so here is the pattern in small pieces.

Register an API client: add it to the `AppFixtures` interface and to the `extend` call.

```
interface AppFixtures {
  projectsApi: ProjectsApi;
}

export const test = base.extend<AppFixtures>({
  projectsApi: async ({ api }, use) => use(new ProjectsApi(api)),
});
```

Register a page object the same way, with the `page` fixture:

```
  projectsPage: async ({ page }, use) => use(new ProjectsPage(page)),
```

Sign in through the API: a token fixture per role.

```
  adminToken: async ({ authApi }, use) => {
    const { email, password } = credentialsFor('admin');
    const { access_token } = await authApi.login(email, password);
    await use(access_token);
  },
```

Start a browser test already signed in: the `signIn` helper in the same file writes the token into `localStorage` before the page loads. Change the storage key (`TOKEN_STORAGE_KEY`) to your application's. If it uses a cookie, call `context.addCookies([...])` instead. If a plain saved session lasts long enough for your application, Playwright's `storageState` is also fine.

A self-cleaning data fixture: this is the pattern to copy for every kind of record.

```
  newProject: async ({ projectsApi, adminToken }, use) => {
    const draft = buildProject();
    await use(draft);
    await projectsApi.deleteByName(adminToken, draft.name);   // runs even when the test
  },

  seededProject: async ({ projectsApi, adminToken, newProject }, use) => {
    await use(await projectsApi.create(adminToken, newProject.name));
  },
```

`newX` gives a unique draft that doesn't exist yet. `seededX` creates it first through the API, so a UI test can start from an existing record. Clean up by an identifier the test can't change (a slug, a name, an email), and say so in a comment.

7. Page objects (`src/pages`)

One class per page or reusable component. Expose actions and state queries. Keep locators private unless a spec needs to assert on them.

```
import { test } from '@playwright/test';
import type { Locator, Page } from '@playwright/test';

export class ProjectsPage {
  readonly rows: Locator;
  private readonly addButton: Locator;

  constructor(private readonly page: Page) {
    this.rows = page.getByRole('row');
    this.addButton = page.getByRole('button', { name: 'New project' });
  }

  async goto(): Promise<void> {
    await this.page.goto('/projects');
  }

  async openNewProjectForm(): Promise<void> {
    await test.step('Open the new project form', async () => {
      await this.addButton.click();
    });
  }
}
```

- Wrap each action in `test.step`. That's what turns the report into a readable story.
- Put shared pieces, such as a header or a side menu, in `src/pages/components` and compose them into pages.
- The admin list base class in `src/pages/admin/admin-list-page.ts` (search box plus table) is reusable if your application has several list screens built the same way.

- Prefer `getByRole`, `getByLabel` and `getByTestId`. Avoid CSS and XPath.
- Never use `waitForTimeout`. Use web-first assertions, or `expect.poll` for something that happens in the API.

8. Write the first tests

Start small and make each step green before the next.

1. **An API smoke test**: log in and read a list. It proves configuration, credentials and the base client work.
2. **A UI smoke test**: sign in and see the landing page. It proves the locators and the browser side work.
3. **One create flow**: create a record through the API, find it in the UI, and check it is removed afterwards.
4. **Then widen**: by feature, in `tests/api` and `tests/ui/<area>`, one feature per file.

```
import { expect, test } from '@fixtures/app.fixture';
import { Tags } from '@utils/tags';

test.describe('Projects', () => {
  test(
    'a new project appears in the list',
    { tag: [Tags.smoke, Tags.ui] },
    async ({ projectsPage, seededProject }) => {
      await projectsPage.goto();

      await expect(projectsPage.rows.filter({ hasText: seededProject.name })).toBeVisibl
    },
  );
});
```

Tag every test with `Tags.smoke` or `Tags.regression`, plus `Tags.api` or `Tags.ui`. The report uses those tags for its labels, and CI selects suites by them.

When you find a bug in the application, don't weaken the assertion and don't leave the build red. Write the correct expectation, add `Tags.knownIssue` and call `test.fail(true, '<what is`

`wrong>')` at the top. It counts as passing while the bug exists and fails when it is fixed. This is also a nice thing to show a client.

9. Reporting

Labels are applied automatically by the `reportMetadata` fixture in `base.fixture.ts`: the epic is `APP_NAME`, the layer comes from the `@api` tag, the feature from the `describe` title and the severity from `@smoke`. You don't touch it unless you want different grouping.

Review `environmentInfo` in `playwright.config.ts`. It shows up in the report header, so add anything a reader would want, such as the build number or the branch.

10. The optional layers

You can drop any of these cleanly. Each one is separate on purpose.

Layer	Where	To remove it
Accessibility	`src/utils/accessibility.ts`, `tests/ui/accessibility`	Delete both, and `@axe-core/playwright` from `package.json`.
Visual	`tests/ui/visual`, the visual step in the workflow	Delete the folder and the step. Otherwise regenerate the baselines (see the README).
Mobile	`tests/ui/mobile`, `mobile-chrome` in the config	Delete the folder and the `mobile-chrome` entries in the config and the workflow matrix.
Database	`src/db`, `tests/db`, `toolshopDb` fixture, `env.db`	Delete them, the `DB_*` variables and `mysql2`.

Keep the accessibility layer. It is quick to adapt and clients like it. For it, change the page list and reset the `knownViolations` baseline so it only holds what your application already has.

If you keep the database layer, `src/db/toolshop-db.ts` is a small read-only class: rename it, change the queries to your schema, and keep it read-only. Create data through the API, then look at what was stored.

11. Running the application under test in CI

This is the part that depends most on your situation. Pick one.

- **A hosted test environment** (a staging URL that is always up). The simplest. Set `UI_URL` and `API_URL` in the workflow, delete the "Start the application under test" step, the QEMU step and `scripts/start-app.mjs`. The risk is that someone else's changes to that environment can break your run.
- **Your own Docker Compose file** for the application. Replace `infra/toolshop/docker-compose.yml`, then edit `scripts/start-app.mjs`: the compose file path, the URLs it waits for, and the seeding commands. Pin images by digest so the application can't change under you. This is what the Toolshop version does.
- **Build the application in the workflow** from its own repository, with a second `actions/checkout`, and then start it. Do this when you test your own product.

Then edit `.github/workflows/playwright.yml`:

- Change the `APP_NAME`, the credential secret names if you changed the roles, and delete the `DB_*` block if you removed the database layer.
- Remove the QEMU step unless an image really is arm64-only.
- Add your credentials as repository secrets (and again under Dependabot secrets if you want Dependabot pull requests to pass).
- The visual step runs inside the Playwright container image, and its tag must match your `@playwright/test` version and the `Dockerfile`. Update all three together.

12. Cleanup checklist before you show it to a client

- [ ] Nothing in `src` or `tests` still mentions Toolshop (search for `Toolshop`, `toolshop`, `practicesoftwaretesting`).
- [ ] `infra/toolshop` is replaced or removed, and `scripts/start-app.mjs` matches.
- [ ] The `docs/images` screenshots and `docs/case-study.md` are replaced with your own.
- [ ] The README describes your application: coverage, how to run, known issues.
- [ ] `tests/ui/visual/__screenshots__` holds your baselines, or the folder is gone.
- [ ] The accessibility `knownViolations` baseline is yours.
- [ ] `.env.local` is git-ignored and no real credential is in the history. Run `git log -p | grep -i password` if unsure.
- [ ] `npm run lint`, `npm run typecheck` and `npm run format:check` pass, and the workflow is green on a clean clone.

13. Habits that keep a suite trustworthy

These are the things that went wrong while building the Toolshop version.

- **Tests that share data fail at random.** Always create unique data and clean it up. Never assert on a total or a stock level that another test can change.
- **Expect surprises from the application, not only your code.** Server-side caches, search that ignores short words, tokens that expire in minutes and statuses that a background job changes later all caused failures. When a test fails strangely, look at the application's behaviour before you change the test.
- **Check the evidence before adding a retry.** Open the trace. A retry that hides a real race is worse than a red build.
- **Don't run too many workers against a small environment.** Set `WORKERS` to what the environment can take.
- **Verify a changed assumption in the data, not in your head.** Query the database or the API and look.
- **Ask before hard-coding application values** that a test could read from the application instead.

Quick map of the repository

```
src/config      environment profiles, credentials, global setup   (keep; edit Role and g
src/api         API clients                                        (replace the domain c
src/data        builders and sample values                         (replace)
src/pages       page objects and components                        (replace)
src/fixtures    base.fixture.ts (keep), app.fixture.ts (rewrite)
src/utils       tags, accessibility helper, contract helper        (keep)
src/db          read-only database queries                         (replace or remove)
tests           your specs                                         (replace)
scripts         start-app.mjs                                      (adapt)
infra           docker-compose for the application                 (replace)
.github         CI workflow, Dependabot, templates                 (adapt)
```